

Group B Practical 2

```
package HarmonicMean;

import java.util.*;

public class HarmonicMean {

    public static void main(String[] args) {
        // Sample documents and queries
        Map<String, String> documents = new HashMap<>();
        documents.put("doc1", "Text analysis is the process of obtaining information from text documents. Text analysis involves various techniques for EXTRACTING documents");
        documents.put("doc2", "Text analysis involves various techniques");

        Map<String, String> queries = new HashMap<>();
        queries.put("query1", "Text analysis");
        queries.put("query2", "Text analysis process");

        // Relevance judgments (queryId, docId, isRelevant)
        Map<String, List<String>> relevanceJudgments = new HashMap<>();
        relevanceJudgments.put("query1", Arrays.asList("doc1"));
        relevanceJudgments.put("query2", Arrays.asList("doc1", "doc2"));

        // Calculate precision, recall, F-measure, and E-measure for each query
        for (String queryId : queries.keySet()) {
            String query = queries.get(queryId);
            List<String> relevantDocuments = relevanceJudgments.getDefault(queryId, new ArrayList<>());

            // Tokenize the query
            Set<String> queryTokens = tokenize(query);
            Set<String> retrievedDocuments = new HashSet<>(documents.keySet());

            // Calculate precision, recall, f-measure, and e-measure
            double precision = calculatePrecision(queryTokens, retrievedDocuments);
            double recall = calculateRecall(new HashSet<>(relevantDocuments),
retrievedDocuments);
            double fMeasure = calculateFMeasure(precision, recall);
            double eMeasure = calculateEMeasure(precision, recall);

            // Print the results
            System.out.println("Query: " + query);
            System.out.println("Relevant Documents: " + relevantDocuments);
        }
    }
}
```

```

        System.out.println("Retrieved Documents: " + retrievedDocuments);
        System.out.println("Precision: " + precision);
        System.out.println("Recall: " + recall);
        System.out.println("F-measure: " + fMeasure);
        System.out.println("E-measure: " + eMeasure);
        System.out.println();
    }
}

// Tokenize the text into a set of words
private static Set<String> tokenize(String text) {
    return new HashSet<>(Arrays.asList(text.toLowerCase().split("\\s+")));
}

// Calculate precision
private static double calculatePrecision(Set<String> queryTokens, Set<String>
documentTokens) {
    if (documentTokens.isEmpty()) {
        return 0.0;
    }

    Set<String> intersection = new HashSet<>(queryTokens);
    intersection.retainAll(documentTokens);
    return (double) intersection.size() / queryTokens.size();
}

// Calculate recall
private static double calculateRecall(Set<String> queryTokens, Set<String>
documentTokens) {
    if (documentTokens.isEmpty()) {
        return 0.0;
    }

    Set<String> intersection = new HashSet<>(queryTokens);
    intersection.retainAll(documentTokens);
    return (double) intersection.size() / queryTokens.size();
}

// Calculate F-measure
private static double calculateFMeasure(double precision, double recall) {
    if (precision == 0.0 && recall == 0.0) {
        return 0.0;
    }
    return (2 * precision * recall) / (precision + recall);
}

```

```
}  
  
// Calculate E-measure  
private static double calculateEMeasure(double precision, double recall) {  
    if (precision == 0.0 && recall == 0.0) {  
        return 0.0;  
    }  
    return (2 * precision * recall) / (precision + recall);  
}  
}
```

Output

Query: Text analysis
Relevant Documents: [doc1]
Retrieved Documents: [doc2, doc1]
Precision: 0.0
Recall: 1.0
F-measure: 0.0
E-measure: 0.0

Query: Text analysis process
Relevant Documents: [doc1, doc2]
Retrieved Documents: [doc2, doc1]
Precision: 0.0
Recall: 1.0
F-measure: 0.0
E-measure: 0.0

=== Code Execution Successful ===